

# TruthLens ■■ — System Architecture & Tech Stack Report

## Comprehensive Technical Documentation & Dependency Analysis for Deepfake Authenticity SaaS Platform

Author: Antigravity Engineering | Project: Truthified / TruthLens | Version: 1.0.0 | Date: July 2026

### 1. Executive Overview & Problem Statement

**TruthLens** (Truthified) is an enterprise-grade AI Video Authenticity and Deepfake Detection SaaS platform. With the rapid rise of synthetic media, generative AI models, face-swapping algorithms, and AI voice cloning, detecting manipulated digital content has become essential for legal, media, security, and verification agencies. TruthLens delivers high-accuracy frame-by-frame analysis, audio-visual synchronicity checks, confidence scoring (0-100%), downloadable PDF compliance audit reports, and instant verification badges for publishing platforms.

**Core Platform Capability:** TruthLens ingests raw video files, extracts spatial frame features and audio cues, computes statistical micro-inconsistencies via Computer Vision routines, stores immutable records, and renders real-time dashboard visual analytics.

### 2. Detailed Technology Stack & Component Mapping

The application is engineered as a decoupled, multi-tiered architecture combining a high-performance Python FastAPI asynchronous backend, a reactive Next.js 14 (TypeScript) frontend, and an integrated Computer Vision processing engine.

| Layer                  | Technologies / Frameworks                                        | Primary Architectural Purpose                                                         |
|------------------------|------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Frontend Layer         | Next.js 14, React 18, TypeScript, Tailwind CSS, Framer Motion    | Server-side rendering, dynamic reactive UI, layout styling, micro-animations          |
| State & Data Fetching  | TanStack React Query, Axios, React Hook Form, Zod                | Async server state management, HTTP requests, schema validation & form control        |
| Backend API Layer      | FastAPI, Python 3.11, Uvicorn (ASGI), Pydantic v2                | High-speed asynchronous REST API routes, request parsing, dependency injection        |
| Database & ORM         | SQLite (dev) / AsyncPG (prod), SQLAlchemy 2.0 (AsyncIO), Alembic | Async relational data persistence, schema ORM mapping, automated migrations           |
| Security & Auth        | PyJWT / Python-Jose, Passlib (Bcrypt), OAuth2 Bearer Tokens      | Secure user authentication, encrypted password hashing, stateless session JWTs        |
| Computer Vision Engine | OpenCV (Headless), NumPy, Pillow (PIL)                           | Frame extraction, spatial matrix operations, noise/texture analysis, image processing |

|                |                                  |                                                                                     |
|----------------|----------------------------------|-------------------------------------------------------------------------------------|
| Infrastructure | Docker, Docker Compose, Aiofiles | Multi-container orchestration, unified dev/prod environments, non-blocking file I/O |
|----------------|----------------------------------|-------------------------------------------------------------------------------------|

### 3. Deep-Dive: Individual Dependencies & Specific Contributions

Below is an exhaustive documentation of every dependency used across the backend and frontend codebases, detailing exactly what each package does and what specific problem it solves in TruthLens.

3.1 Backend Dependencies (Python / FastAPI Ecosystem)

| Package Name                 | Exact Role & What It Helps In TruthLens                                                                                                                                                                                             |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| fastapi                      | <b>Core Web Framework:</b> Serves as the backbone of the backend API. Enables ultra-fast, asynchronous request handling using Python type hints, automatic OpenAPI/Swagger documentation (`/docs`), and request body serialization. |
| uvicorn[standard]            | <b>Lightning-Fast ASGI Server:</b> Runs the asynchronous FastAPI application server. Equipped with `uvloop` and `httptools` for handling high concurrency and parallel async HTTP video stream requests.                            |
| sqlalchemy[asyncio]          | <b>Asynchronous ORM &amp; DB Engine:</b> Manages database entities (Users, Video Analysis, Reports, Badges) cleanly. Uses async/await syntax to execute database queries concurrently without blocking the main event loop.         |
| aiosqlite                    | <b>Async SQLite Storage Driver:</b> Provides async disk I/O driver for SQLite database (`truthlens.db`) during development and local testing, ensuring zero blocking calls during DB reads/writes.                                  |
| asyncpg                      | <b>Production PostgreSQL Driver:</b> High-performance native PostgreSQL driver designed for async operation, ensuring seamless transition from local SQLite to enterprise PostgreSQL in production.                                 |
| alembic                      | <b>Database Version Control &amp; Migrations:</b> Tracks relational schema evolutions over time. Generates revision scripts to safely upgrade or downgrade database tables without data loss.                                       |
| python-jose[cryptography]    | <b>JWT Token Issuer &amp; Verifier:</b> Implements JSON Web Signature (JWS) and JSON Web Tokens (JWT). Signs user access tokens with secret keys and validates user identities on protected API routes.                             |
| passlib[bcrypt]              | <b>Password Cryptography &amp; Hashing:</b> Securely hashes user account passwords using salt-augmented `bcrypt` hashing before persisting to the database, defending against rainbow table attacks.                                |
| python-multipart             | <b>Multipart Form Parser:</b> Enables FastAPI to handle binary file uploads (`multipart/form-data`). Crucial for accepting large video file uploads from the frontend dropzone.                                                     |
| pydantic & pydantic-settings | <b>Data Validation &amp; Config Parsing:</b> Enforces strict data models for API payloads, response schemas, and reads `.env` variables into structured, validated configuration objects.                                           |
| aiofiles                     | <b>Non-Blocking File I/O:</b> Provides asynchronous file read/write operations for storing uploaded raw videos and generated PDF reports on the server filesystem without stalling worker threads.                                  |
| python-dotenv                | <b>Environment Configuration Loader:</b> Automatically reads `.env` files and populates `os.environ` so secret keys, DB URLs, and CORS origins are kept out of source control.                                                      |
| opencv-python-headless       | <b>Computer Vision &amp; Frame Extraction:</b> Open-source CV library compiled without GUI dependencies. Used for video decoding, frame-by-frame extraction, face crop detection, edge analysis, and pixel variance calculations.   |
| numpy                        | <b>Numerical &amp; Matrix Computations:</b> Operates on multidimensional image array matrices extracted by OpenCV. Computes spatial noise distributions, frame difference metrics, and statistical manipulation scores.             |

|              |                                                                                                                                                                                                   |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| pillow (PIL) | <b>Image Preprocessing &amp; Thumbnail Generation:</b> Handles image conversions, cropping key suspicious frames, generating video preview thumbnails, and preparing images for visual reporting. |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### 3.2 Frontend Dependencies (Next.js / React Ecosystem)

| Package Name          | Exact Role & What It Helps In TruthLens Frontend                                                                                                                                           |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| next (v14.2)          | <b>Full-Stack React Framework:</b> Utilizes App Router architecture, Server-Side Rendering (SSR), static optimization, asset routing, and automatic code-splitting for fast load times.    |
| react & react-dom     | <b>UI Component Engine:</b> Powers the component-driven interface, allowing modular composition of forms, dashboards, analytics charts, upload widgets, and interactive modals.            |
| typescript (v5.6)     | <b>Static Type Safety:</b> Enforces end-to-end type definitions for API responses, user inputs, and props, preventing runtime errors and streamlining developer experience.                |
| tailwindcss & postcss | <b>Utility-First CSS System:</b> Provides responsive, modern styling classes. Enables rapid building of dark/light themes, sleek card layouts, custom glassmorphism, and responsive grids. |
| framer-motion         | <b>Fluid UI &amp; Micro-Animations:</b> Adds smooth spring physics, fade-in transitions, drag-and-drop file upload animations, score gauge meters, and modal motion effects.               |
| @tanstack/react-query | <b>Async Server State Manager:</b> Handles API caching, background re-fetching, loading states, and optimistic UI updates for video upload progress and analysis dashboard polling.        |
| axios                 | <b>HTTP API Client:</b> Formats and sends HTTP requests to the FastAPI backend. Configured with interceptors to automatically attach JWT authorization headers to secure requests.         |
| recharts              | <b>Data Visualization Charts:</b> Renders interactive authenticity confidence graphs, frame anomaly timelines, and voice visual spectrum charts on the dashboard and report pages.         |
| lucide-react          | <b>Modern SVG Icon Library:</b> Renders clean, accessible UI icons for navigation, file status indicators (Shield, Alert, Check, FileText, Upload), and platform trust badges.             |
| react-hook-form       | <b>Performant Form Management:</b> Manages form input state (login, registration, settings) efficiently without triggering full parent component re-renders.                               |
| zod                   | <b>Type-Safe Schema Validation:</b> Defines strict client-side validation rules (email format, password complexity, file constraints) before form submission.                              |
| @hookform/resolvers   | <b>Zod Form Bridge:</b> Seamlessly connects Zod validation schemas with React Hook Form for instant inline error messaging.                                                                |
| react-hot-toast       | <b>User Notifications:</b> Displays elegant floating toast alerts for successful uploads, authentication notifications, error callouts, and download confirmations.                        |
| clsx & tailwind-merge | <b>Dynamic Class Merger:</b> Safely combines dynamic conditional CSS class names without class specificity conflicts or duplicate Tailwind rules.                                          |

## 4. End-to-End System Processing Pipeline

- **Step 1: Ingestion & Upload** — User selects a video file via Next.js Framer Motion drag-and-drop dropzone. Client validates file format/size with Zod and transmits chunked multipart data via Axios to `/api/v1/videos/upload``.
- **Step 2: Non-Blocking Disk Storage** — FastAPI receives the video stream and writes the raw file to server disk using ``aiofiles`` while persisting metadata to ``truthlens.db`` via SQLAlchemy async session.

- **Step 3: Computer Vision Extraction** — Background pipeline triggers `OpenCV` to extract sequential video frames, face bounding boxes, and audio tracks. `NumPy` computes matrix pixel variance and frame-to-frame noise anomalies.
- **Step 4: Authenticity & Confidence Scoring** — Statistical anomalies (facial edge distortion, lip-sync latency, color space inconsistencies) are aggregated into a composite Authenticity Score (0 - 100%).
- **Step 5: Report & Trust Badge Generation** — System generates a cryptographic hash of the analysis result, creates exportable PDF audit reports, and provisions verified trust badge URLs.
- **Step 6: Dashboard Visualization** — React Query fetches updated scan results, and `Recharts` visualizes timeline anomalies for user review.

## 5. Deployment & Container Architecture

TruthLens uses a containerized deployment model via **Docker** and **Docker Compose**. The application is separated into isolated container microservices:

| Container Service  | Base Image & Configuration                                          | Exposed Port & Networking                     |
|--------------------|---------------------------------------------------------------------|-----------------------------------------------|
| truthlens-backend  | Python 3.11-slim, OpenCV headless dependencies, Uvicorn ASGI server | Port `8000` (Internal REST & Swagger `/docs`) |
| truthlens-frontend | Node.js 18-alpine, Next.js production standalone build              | Port `3000` (User Web Application)            |

## 6. Architectural Advantages & Conclusion

The technical architecture of **TruthLens** ensures enterprise scalability, high visual appeal, and strict security compliance. By leveraging Next.js 14 and React 18 on the frontend, users experience fluid interactive performance, while FastAPI and Python's robust Computer Vision tools provide fast, non-blocking media analysis. The stack components work in total harmony to deliver trustworthy, verifiable deepfake detection for modern media applications.